Cliente PDFer — explicação do código

Oct 12, 2026 ·

O `pdfer_client.py` é o cliente de carga do trabalho: faz pedidos POST, GET e de listagem ao servidor PDFer, mede cada um do lado do cliente, guarda também os tempos que o servidor devolve e escreve tudo num CSV, podendo ainda publicar as métricas para o Prometheus. É um único ficheiro em Python 3, sem dependências externas, que corre no node2.

Para que serve

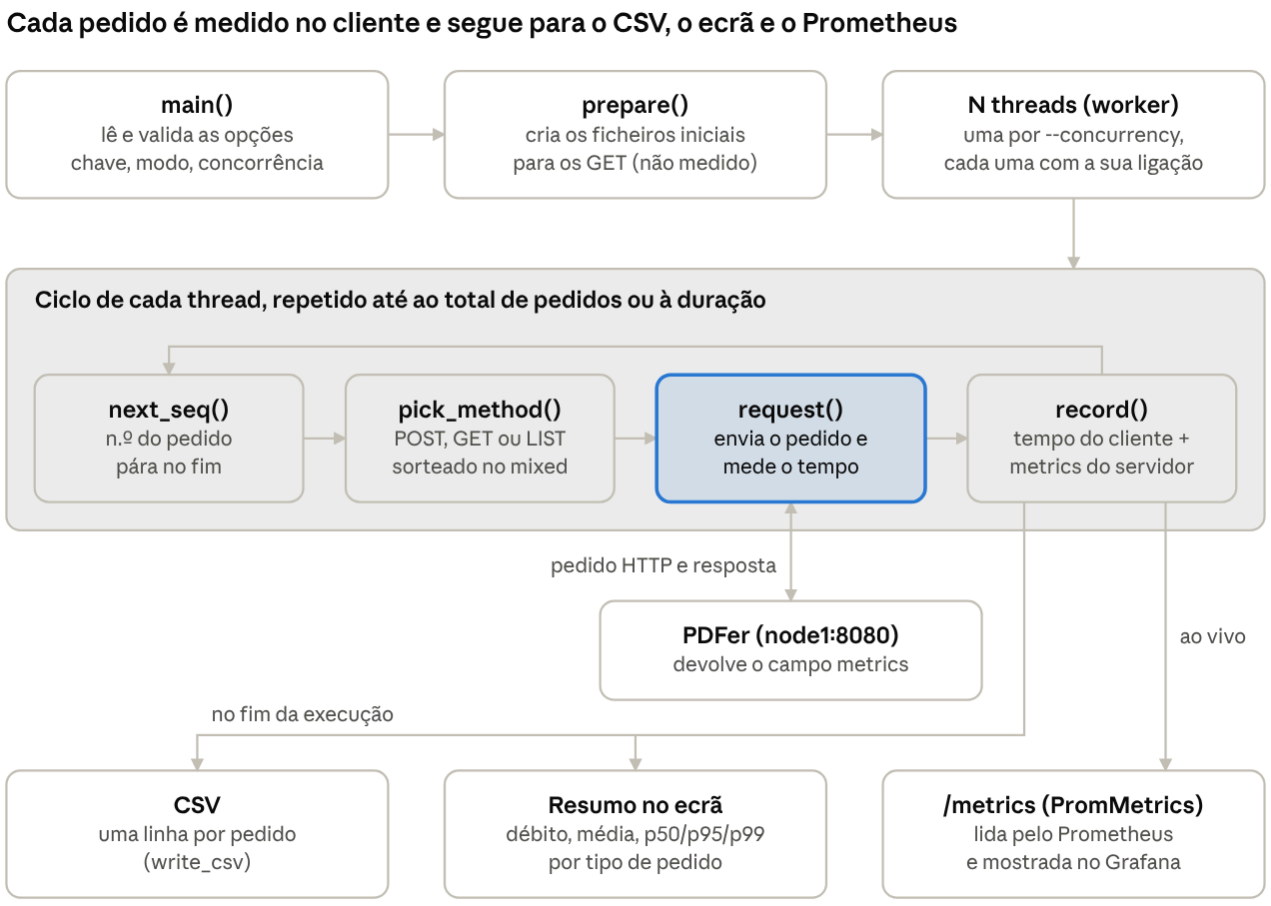
O cliente tem duas funções: pôr o servidor a trabalhar de forma controlada (gerar carga) e medir o desempenho tal como um utilizador o sentiria. O objeto de estudo é o servidor no node1; o cliente é a ferramenta de medição e corre no node2, para não competir com o servidor pelos mesmos recursos.

Requisito do enunciado	Como o cliente o cumpre
Medir o POST e o GET separadamente (obrigatório)	Cada pedido é classificado como `POST`, `GET` ou `LIST` e o resumo, o CSV e as métricas Prometheus separam os três
Tempo de resposta na perspetiva do cliente	Cronómetro à volta de cada pedido, do envio até à resposta completa
Número de objetos por unidade de tempo	Débito em pedidos por segundo, com concorrência configurável
Tempo de geração de um objeto no servidor	Guarda as fases devolvidas pelo PDFer no campo `metrics` (`generateFile`, `encryption`, …)
Integração com o Prometheus (Nota 2, valorizada)	Opção `--metrics-port` publica as métricas no formato do Prometheus
Ambiente sem interface gráfica	Programa de linha de comandos, lançado por SSH

O servidor PDFer tem três operações, e o cliente usa as três:

Nome no cliente	Pedido HTTP	Corpo (JSON)	O que o servidor faz
`POST`	`POST /files`	`key`, `length`, `fileName`	Gera um texto de `length` caracteres, cifra-o (AES-GCM) e guarda-o na pasta `store`
`GET`	`GET /files/{fileId}`	`key`	Lê o ficheiro, decifra-o e devolve o texto
`LIST`	`GET /files`	—	Lista os nomes dos ficheiros guardados

Fluxo de execução



fluxo de uma execução do cliente · preparação, ciclo de cada thread e saídas

Depois de validar as opções e preparar os ficheiros, cada thread repete o mesmo ciclo até se atingir o total de pedidos ou a duração. As métricas para o Prometheus são atualizadas a cada pedido; o CSV e o resumo são escritos quando a execução termina.

Como se usa

Corre-se no node2 com `python3 pdfer_client.py [opções]`. Todas as opções têm um valor por omissão, e `--help` mostra a lista completa.

Opção	Omissão	Para que serve	Experiência
`--host`, `--port`	192.168.57.11, 8080	Endereço do servidor	—
`--mode`	`post`	`post`, `get`, `list` ou `mixed`	E6
`--length`	1000	Caracteres de cada ficheiro criado	E1, E7
`--requests`	100	Pedidos medidos (sem contar o aquecimento)	Todas
`--duration`	—	Em vez de `--requests`, correr durante N segundos	E2
`--concurrency`	1	Pedidos em simultâneo (threads)	E2
`--warmup`	5	Pedidos iniciais excluídos do resumo	Todas
`--prepare`	20 em `get`/`mixed`, 0 em `post`/`list`	Ficheiros criados antes de medir	E3, E5
`--post-ratio`, `--list-ratio`	0,5 e 0	Proporção de POST e de listagens no modo `mixed`	E6
`--key`	`chave-de-teste16`	Chave AES (16, 24 ou 32 caracteres)	—
`--seed`	42	Semente aleatória, para repetir a mesma sequência	Todas
`--output`	nome automático	CSV de saída; se existir, acrescenta linhas	Todas
`--metrics-port`	desligado	Porta da página `/metrics` para o Prometheus (ex.: 9200)	Visualização
`--metrics-linger`	20 s	Tempo que `/metrics` fica ativa no fim	Visualização
`--timeout`	120 s	Tempo máximo de cada pedido	—

Exemplos:

```
# um único POST, sem aquecimento
python3 pdfer_client.py --mode post --length 1000 --requests 1 --warmup 0

# E1: tempo em função do tamanho (repetir para cada --length)
python3 pdfer_client.py --mode post --length 10000 --requests 100 --output
e1.csv

# E2: débito com 8 pedidos em simultâneo durante 60 s
python3 pdfer_client.py --mode get --concurrency 8 --duration 60 --output
e2.csv

# E3: listagem com 1000 ficheiros na pasta
python3 pdfer_client.py --mode list --prepare 1000 --length 100 --requests
100 --output e3.csv

# E6: carga mista vista ao vivo no Grafana
python3 pdfer_client.py --mode mixed --post-ratio 0.3 --list-ratio 0.1 \
    --duration 120 --concurrency 4 --metrics-port 9200
```

O programa valida as opções antes de enviar qualquer pedido. Uma chave com tamanho inválido, por exemplo, é recusada logo, em vez de dar origem a centenas de pedidos falhados.

Estrutura do código

O ficheiro tem cerca de 520 linhas, organizadas em quatro blocos: métricas para o Prometheus, comunicação com o servidor, execução da carga e programa principal. Só usa a biblioteca padrão do Python 3 (`http.client` , `threading` , `csv` , `json` , `argparse` , `statistics` , `http.server`).

Componente	Tipo	Responsabilidade
`PromMetrics`	classe	Guarda histogramas, contadores e indicadores em memória e gera o texto no formato do Prometheus (`render`)
`start_metrics_server`	função	Arranca, numa thread à parte, um pequeno servidor HTTP que responde em `/metrics`
`PdferConnection`	classe	Uma ligação HTTP persistente por thread, com `TCP_NODELAY` ; envia o pedido e mede o tempo

Componente	Tipo	Responsabilidade
`do_post`, `do_get`, `do_list`	funções	Montam cada pedido no formato que o PDFer espera
`parse_server_metrics`, `files_listed`	funções	Tiram da resposta JSON os tempos do servidor e o n.º de ficheiros listados
`LoadRun`	classe	Coordena a execução: preparação, threads de trabalho, registo, CSV e resumo
`LoadRun.prepare`	método	Cria os ficheiros iniciais de que os GET precisam (fase não medida)
`LoadRun.worker`	método	Ciclo de cada thread: escolher o pedido, enviar, medir, registar
`next_seq`, `pick_method`, `pick_existing`	métodos	Partilham estado entre threads com um cadeado (n.º do pedido, tipo de pedido, ficheiro a ler)
`LoadRun.record`	método	Junta o tempo do cliente e os do servidor numa linha e atualiza as métricas Prometheus
`LoadRun.write_csv`, `LoadRun.summary`	métodos	Escrevem o CSV e o resumo no ecrã
`pct`	função	Calcula um percentil por interpolação linear
`main`	função	Lê e valida as opções, corre a carga e mantém `/metrics` ativa no fim

Pormenores de implementação

Ligação persistente e sem atraso de Nagle

Cada thread abre uma ligação TCP ao servidor e reutiliza-a em todos os pedidos (keep-alive), como faz um browser. Abrir uma ligação nova por pedido somaria ao tempo medido um custo que não é do servidor.

```
self.conn = http.client.HTTPConnection(self.host, self.port,
timeout=self.timeout)
self.conn.connect()
self.conn.sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
```

O `TCP_NODELAY` desliga o algoritmo de Nagle, que junta pacotes pequenos e pode atrasá-los cerca de 40 ms. Nos testes, este efeito acrescentava 40 ms constantes a pedidos que demoravam 2 ms. Se a ligação persistente cair, o pedido é repetido uma vez com uma ligação nova.

Medição do tempo

```
t0 = time.perf_counter()
self.conn.request(method, path, body=body, headers=headers)
resp = self.conn.getresponse()
data = resp.read()
elapsed_ms = (time.perf_counter() - t0) * 1000.0
```

`perf_counter()` é o relógio mais preciso do Python para intervalos curtos. O cronómetro começa antes do envio e pára depois de ler a resposta inteira, por isso num GET de um ficheiro grande a transferência dos dados também conta, como contaria para um utilizador. A diferença entre este tempo e o `total` devolvido pelo servidor é o tempo de rede e de espera em fila.

Concorrência

Com `--concurrency N` criam-se N threads, cada uma com a sua ligação: N "utilizadores" a fazer pedidos ao mesmo tempo. O estado partilhado (n.º do próximo pedido, lista de ficheiros criados, gerador aleatório) é protegido por um cadeado, para dois trabalhadores não receberem o mesmo n.º nem lerem a lista enquanto outro a altera.

```
with self.lock:
    self.seq += 1
    return self.seq
```

As threads servem bem aqui porque passam quase todo o tempo à espera da rede; o limite do Python a uma thread de cada vez (GIL) só pesaria se o cliente fizesse muito cálculo. Mesmo assim, convém vigiar o CPU do node2 durante os testes: se chegar perto de 100%, o limite medido é do cliente e não do servidor.

Escolha do pedido e dos ficheiros

Nos modos `post`, `get` e `list` todos os pedidos são do mesmo tipo. No modo `mixed`, cada pedido é sorteado: com `--post-ratio 0.3 --list-ratio 0.1`, cerca de 30% são POST, 10% listagens e 60% GET. Os GET leem um ficheiro sorteado entre os já criados (pelo `--prepare` ou por POST anteriores da mesma execução). A semente fixa (`--seed`) faz com que a mesma experiência repita a mesma sequência de escolhas.

Os nomes dos ficheiros incluem a execução e o n.º do pedido, por exemplo `cli-20261008-175000-000012.txt`, para nunca se repetirem entre execuções.

Aquecimento

```
"phase": "warmup" if seq <= self.a.warmup else "measure",
```

Os primeiros pedidos (5 por omissão) ficam marcados como `warmup`: continuam no CSV mas não entram no resumo. Nos testes ao PDFer, a primeira listagem demorou 3795 µs e a seguinte 126 µs, porque no início a informação da pasta ainda não estava em memória (cache fria).

Erros

Um pedido que falha (ligação recusada, timeout, resposta HTTP 4xx ou 5xx) fica registado com a mensagem na coluna `error` e o programa continua. Os erros são um resultado: quando o servidor começa a falhar, encontrou-se o seu limite. O resumo mostra a percentagem de erros por tipo de pedido.

Percentis

Os tempos são ordenados e o percentil p é o valor na posição (n − 1) × p / 100, interpolando entre os dois vizinhos quando a posição não é inteira. Com os tempos 2, 3, 3, 4 e 50 ms, a média é 12,4 ms, a mediana 3 ms e o p99 cerca de 48 ms: a média esconde que um pedido foi muito lento, por isso o resumo mostra os três.

Resultados produzidos

Cada execução produz três saídas: um CSV com uma linha por pedido, um resumo no ecrã e, com `--metrics-port`, uma página `/metrics` para o Prometheus.

CSV

Sem `--output`, o nome é automático, por exemplo `resultados-post-1000-c1-20261008-175000.csv` (modo, `length`, concorrência, data e hora). Com `--output e1.csv`, várias execuções acrescentam linhas ao mesmo ficheiro, o que junta as repetições de uma experiência; o `run_id` distingue-as.

Coluna	Conteúdo
`run_id`, `seq`, `worker`	Execução, n.º do pedido e thread que o fez
`phase`	`warmup` ou `measure`
`method`	`POST`, `GET` ou `LIST`
`length`, `file_name`	Tamanho e nome do ficheiro (vazios nas listagens)
`files_listed`	N.º de ficheiros devolvidos (só nas listagens)
`start_epoch`	Início do pedido em segundos Unix, para cruzar com os gráficos do Prometheus
`client_ms`	Tempo medido pelo cliente, em ms
`http_status`, `ok`, `error`	Código HTTP, 1 se correu bem, mensagem de erro
`response_bytes`	Tamanho da resposta
`srv_<fase>_us`	Tempos devolvidos pelo servidor, em µs: `total`, `readFileSystemEntries`, `generateFile`, `encryption`, `writeFile`, `readFile`, `decryption`

Resumo no ecrã

Para cada tipo de pedido, sem os de aquecimento: n.º de pedidos e de erros, débito em pedidos por segundo, tempo do cliente (média, p50, p95, p99, máximo) e a média de cada fase do servidor. Nas listagens mostra também quantos ficheiros a pasta tinha.

Métricas para o Prometheus

Com `--metrics-port 9200`, o cliente responde em `http://192.168.57.12:9200/metrics` no formato de texto do Prometheus. A classe `PromMetrics` implementa esse formato diretamente, sem a biblioteca `prometheus_client`, para o cliente continuar sem dependências.

Métrica	Tipo	Etiquetas	Conteúdo
`pdfer_client_request_duration_seconds`	histograma	`method`	Tempo de resposta visto pelo cliente
`pdfer_server_phase_duration_seconds`	histograma	`method`, `phase`	Tempo de cada fase reportado pelo servidor
`pdfer_client_requests_total`	contador	`method`, `result`	Pedidos feitos, `ok` ou `error`
`pdfer_client_files_listed`	indicador	—	Ficheiros na última listagem
`pdfer_client_running`	indicador	—	1 enquanto gera carga, 0 depois
`pdfer_client_run_info`	indicador	`run_id`, `mode`, `length`, `concurrency`	Parâmetros da execução (valor sempre 1)

Os histogramas usam 16 limites, de 0,5 ms a 60 s, o que permite ao Prometheus calcular percentis com `histogram_quantile`. Como a página só existe enquanto o cliente corre, o programa mantém-na ativa mais 20 s no fim (`--metrics-linger`), para o Prometheus recolher os últimos valores.

O CSV e o Prometheus complementam-se: o CSV guarda cada pedido e serve a análise estatística (percentis exatos, intervalos de confiança); o Prometheus agrega de 5 em 5 segundos e serve para ver ao vivo, no Grafana, o tempo de resposta ao lado do CPU, disco e rede do node1.

Validação

O cliente foi testado contra um servidor de imitação com as mesmas rotas e o mesmo formato de resposta do PDFer, e depois contra um Prometheus real.

Teste	Resultado
POST, GET e listagem, sozinhos e em modo misto, com 1 a 4 threads	0 erros; os três tipos aparecem separados no resumo e no CSV
Chave com 5 caracteres	Recusada antes de enviar qualquer pedido
Servidor desligado	100% de erros registados (`ConnectionRefusedError`), sem o programa parar
Tempo do cliente com e sem resposta enviada em dois pedaços	44 ms contra 2,8 ms para o mesmo servidor: era o efeito de Nagle, eliminado com `TCP_NODELAY` e uma resposta num só pedaço (como faz o PDFer, escrito em Go)
Página `/metrics` lida pelo parser oficial do Prometheus	6 famílias de métricas válidas, durante e depois da carga
Recolha por um Prometheus 3.5 real	Alvo `UP`; pedidos/s, p95 por tipo e média de cada fase do servidor calculados corretamente

No PDFer real, os primeiros testes com `curl` confirmaram a API e o formato do campo `metrics` em que o cliente se baseia.

Limitações e melhorias possíveis

- **Carga em ciclo fechado.** Cada thread só envia um pedido quando o anterior termina. Quando o servidor fica lento, o cliente também abranda e não mostra a fila que se formaria com utilizadores independentes. Uma melhoria seria um modo de taxa fixa (pedidos por segundo) em vez de concorrência fixa.
- **Um único processo Python.** Chega para a carga que um node1 com 1 vCPU aguenta, mas deve confirmar-se, no Grafana, que o CPU do node2 não está no limite durante os testes.
- **Os ficheiros acumulam-se.** O PDFer não tem operação para apagar, por isso a pasta `store` tem de ser limpa no node1 entre experiências (`sudo rm ~/pdfer-lab/store/cli-*`).
- **Percentis no Prometheus são aproximados.** O `histogram_quantile` interpola dentro dos limites dos histogramas; os valores exatos são os do CSV.
- **Sem verificação do conteúdo.** O cliente não confirma que o texto devolvido pelo GET tem o tamanho pedido. Seria uma verificação simples a acrescentar.

- **A medição tem custo.** Durante os testes correm também o node_exporter e o cAdvisor no node1, o que consome uma parte do único vCPU do servidor e deve ser referido como limitação das experiências.